Introduction aux Principes Fondamentaux du ML

Objectif du TP :

Dans ce TP, nous allons explorer les deux grands paradigmes de l'apprentissage automatique :

- **L'apprentissage supervisé**, à travers l'utilisation d'un algorithme de régression linéaire simple (modèle de scikit-learn), ainsi que d'une régression logistique.
- **L'apprentissage non supervisé**, à travers l'implémentation de l'algorithme K-means pour le clustering, et sa comparaison avec l'implémentation existante dans scikit-learn.

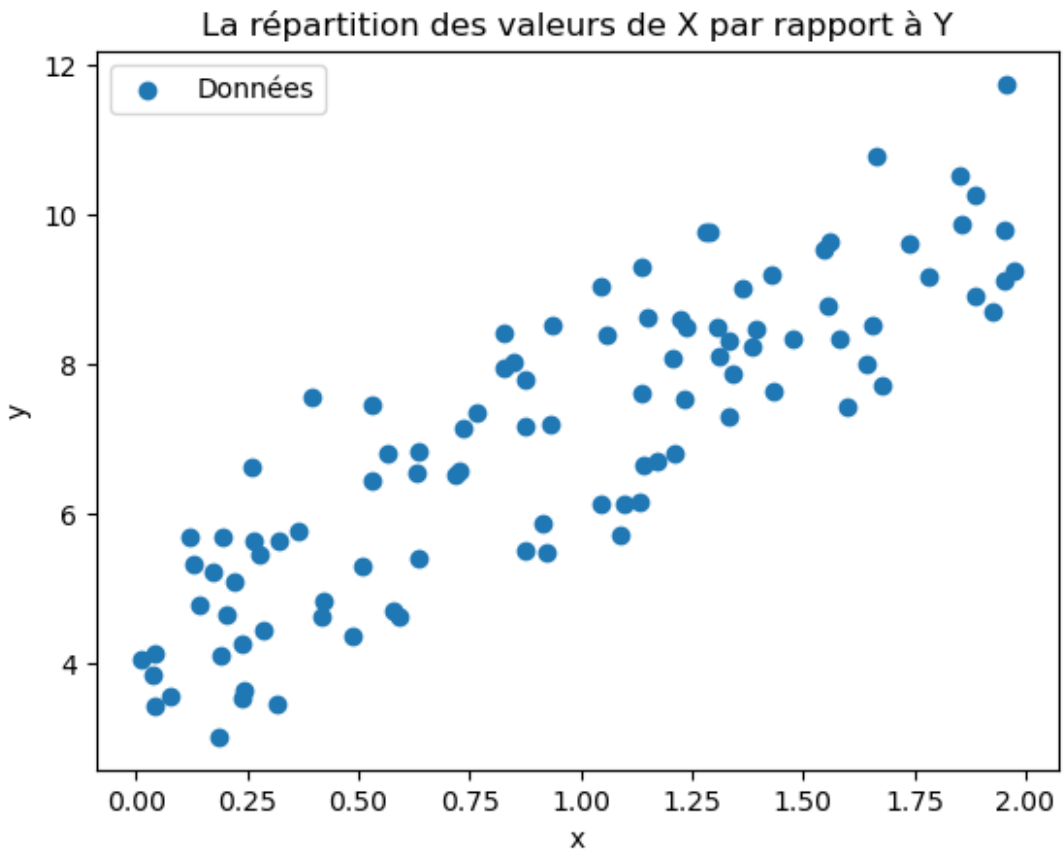
Partie 1 : Apprentissage supervisé – Régression linéaire simple

Génération du jeu de données (xi,yi)

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Générer un dataset jouet
np.random.seed(0)
X = 2 * np.random.rand(100, 1)   # x dans [0,2]
y = 4 + 3 * X + np.random.randn(100, 1)  # y = 4 + 3x + bruit
```

Visualisation des données : la répartition des valeurs de X en fonction de Y..

```
plt.scatter(X, y, label="Données")
plt.title("La répartition des valeurs de X par rapport à Y")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

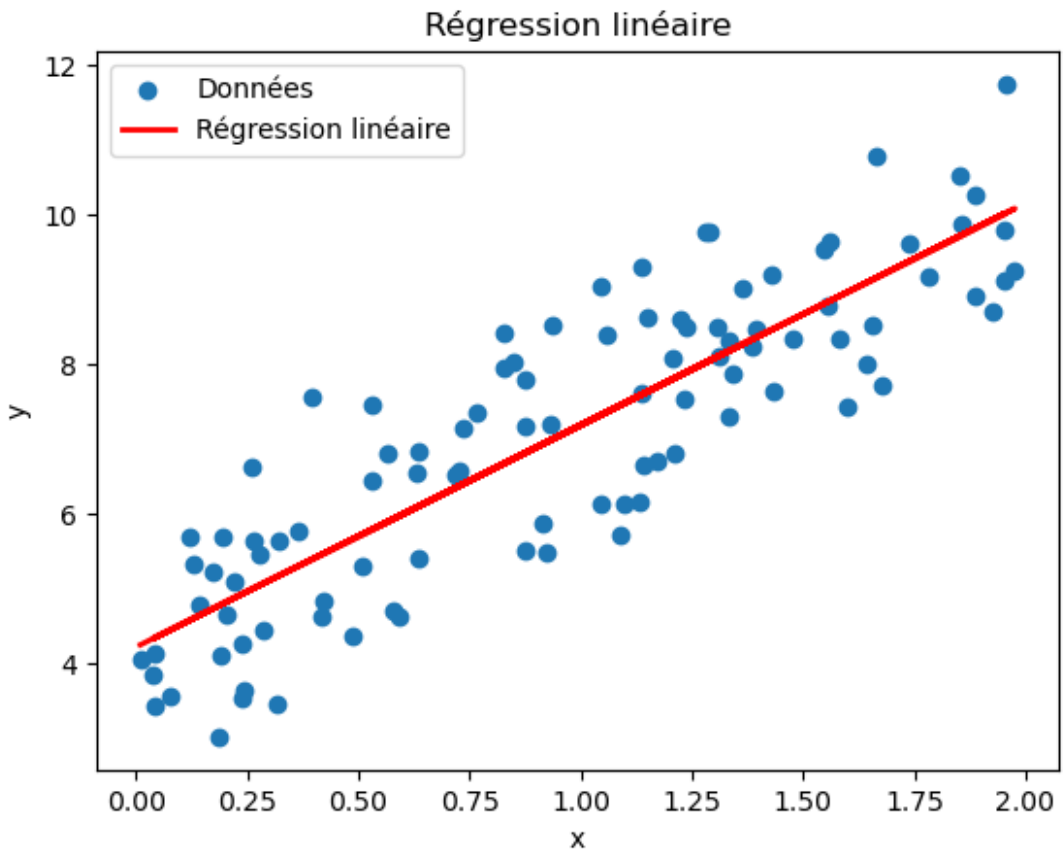


Utilisation de scikit-learn : Entraînement du modèle de régression linéaire.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)
print("Score d'entrainement : ", model.score(X, y))

Score d'entrainement :  0.7469629925504755
```

Traçage de la droite de régression :

```
plt.scatter(X, y, label="Données")
plt.plot(X, model.predict(X), color="red", linewidth=2,
label="Régression linéaire")
plt.title("Régression linéaire")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```



Évaluation du modèle : Calculez l'erreur quadratique moyenne (MSE) entre les valeurs réelles et les valeurs prédites.

```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error

y_pred = model.predict(X)

mse = mean_squared_error(y, y_pred)
mae = mean_absolute_error(y, y_pred)


print(f"L'erreur quadratique moyenne (MSE)  : {mse:.4f}")
print(f"L'erreur absolue moyenne (MAE) : {mae:.4f}")

L'erreur quadratique moyenne (MSE)  : 0.9924
L'erreur absolue moyenne (MAE) : 0.8493
```

Partie 2 : Apprentissage supervisé – Régression logistique

Chargement et exploration du jeu de données

```
from sklearn.datasets import load_iris
import pandas as pd

# Charger le dataset Iris
iris = load_iris()

# Création d'un DataFrame
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Affichage des premières lignes
X.head()
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width
(cm)
0                5.1               3.5                1.4
0.2
1                4.9               3.0                1.4
0.2
2                4.7               3.2                1.3
0.2
3                4.6               3.1                1.5
0.2
4                5.0               3.6                1.4
0.2
```

```
print("Nombre d'échantillons :", X.shape[0])
print("Nombre de caractéristiques :", X.shape[1])
print("Classes :", iris.target_names)
```

```
Nombre d'échantillons : 150
Nombre de caractéristiques : 4
Classes : ['setosa' 'versicolor' 'virginica']
```

```
y
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
```

```
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Réponse aux questions

- Il y a 150 échantillons et 4 caractéristiques.
- Il existe 3 classes (setosa, versicolor, virginica).
- Il s'agit donc d'un problème de classification multi-classes.

Préparation des données

Nous sélectionnons deux caractéristiques pour faciliter la visualisation.

```
# Sélection de deux caractéristiques
X_selected = X[['petal length (cm)', 'petal width (cm)']]
X_selected
```

```
     petal length (cm)  petal width (cm)
0                  1.4               0.2
1                  1.4               0.2
2                  1.3               0.2
3                  1.5               0.2
4                  1.4               0.2
..                 ...               ...
145                5.2               2.3
146                5.0               1.9
147                5.2               2.0
148                5.4               2.3
149                5.1               1.8

[150 rows x 2 columns]
```

Séparation en ensembles d'entraînement et de test :

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_selected, y,
test_size=0.3, random_state=42)
```

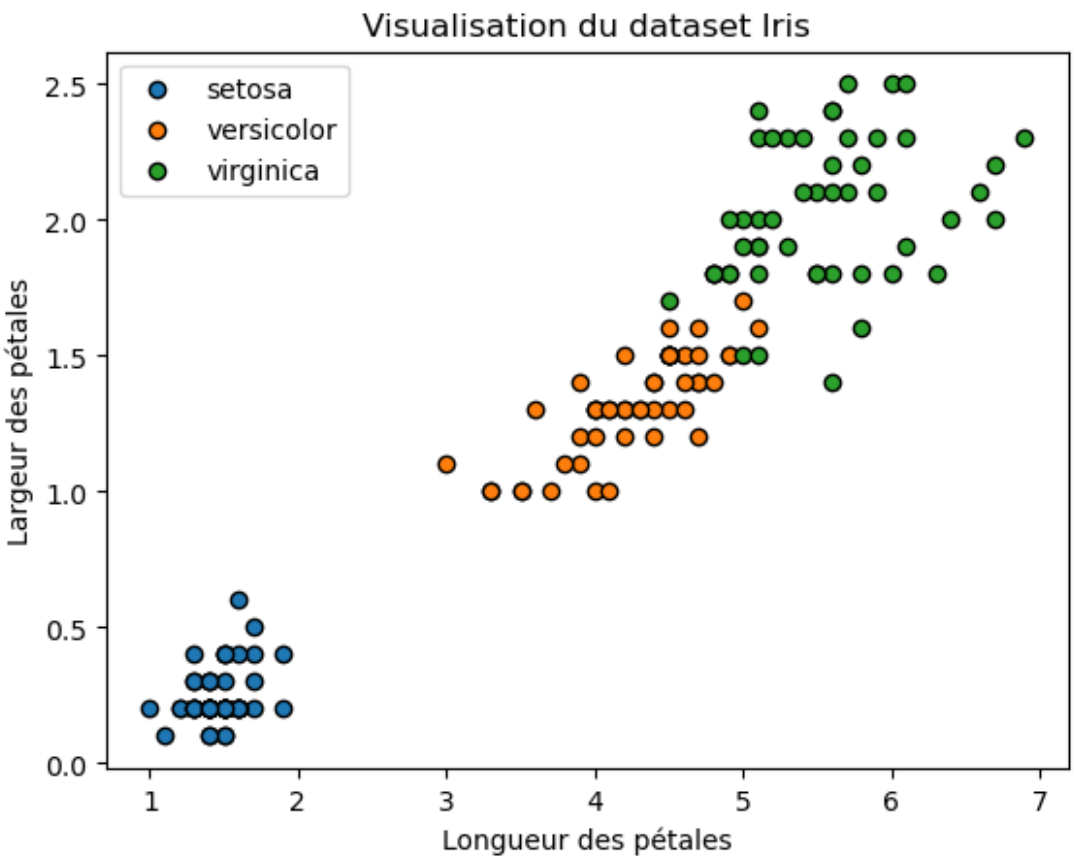
Réponse :

La séparation train/test permet d'évaluer la capacité de généralisation du modèle sur des données jamais vues.

Visualisation des données

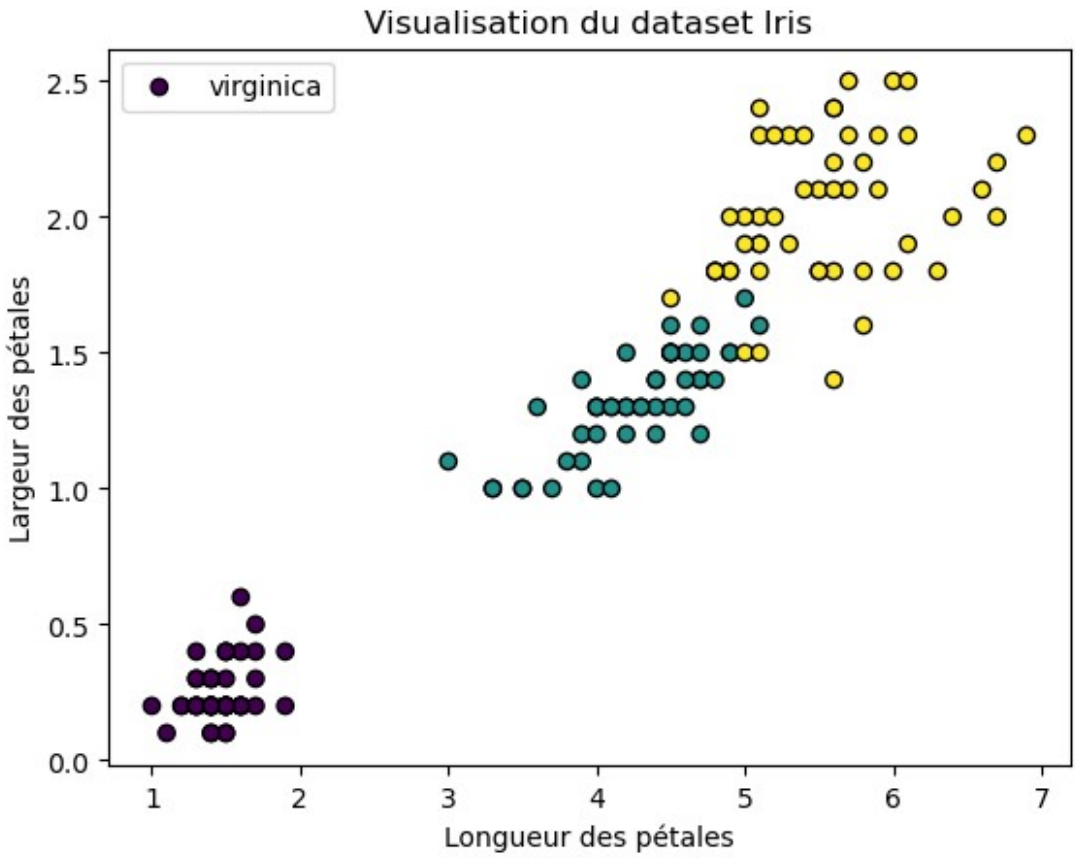
```
import matplotlib.pyplot as plt
```

```
plt.figure()
plt.title("Visualisation du dataset Iris")

for i, label in enumerate(iris.target_names):
    X0 = X_selected.iloc[y == i, 0]
    X1 = X_selected.iloc[y == i, 1]
    plt.scatter(X0, X1, label=label, edgecolor='k')

plt.xlabel("Longueur des pétales")
plt.ylabel("Largeur des pétales")
plt.legend()
plt.show()
```




```
import matplotlib.pyplot as plt

plt.figure()
plt.title("Visualisation du dataset Iris")
plt.scatter(X_selected.iloc[:,0], X_selected.iloc[:,1], label=label,
edgecolor='k',c=y)

plt.xlabel("Longueur des pétales")
```

```
plt.ylabel("Largeur des pétales")
plt.legend()
plt.show()
```



Entraînement du modèle (scikit-learn)

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
print("Score d'entrainement : ", model.score(X_train, y_train))

Score d'entrainement :  0.9523809523809523
```

Prédiction et évaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix

# Prédictions
y_pred = model.predict(X_test)
```

```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy :", accuracy)
```

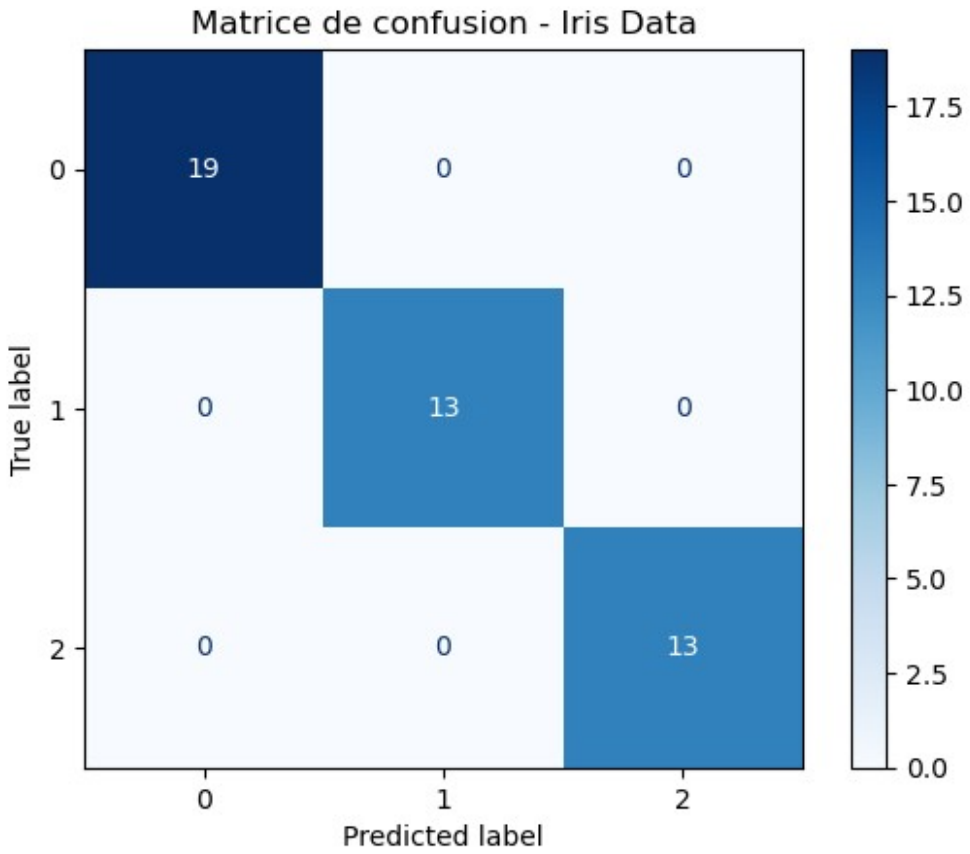
```
Accuracy : 1.0
```

```python
# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print("Matrice de confusion :\n", cm)
```

```
Matrice de confusion :
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

```python
from sklearn.metrics import ConfusionMatrixDisplay

plt.figure(figsize=(3,2))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=np.unique(y))
disp.plot(cmap=plt.cm.Blues)
plt.title(f"Matrice de confusion - Iris Data")
plt.show()
```

```
<Figure size 300x200 with 0 Axes>
```

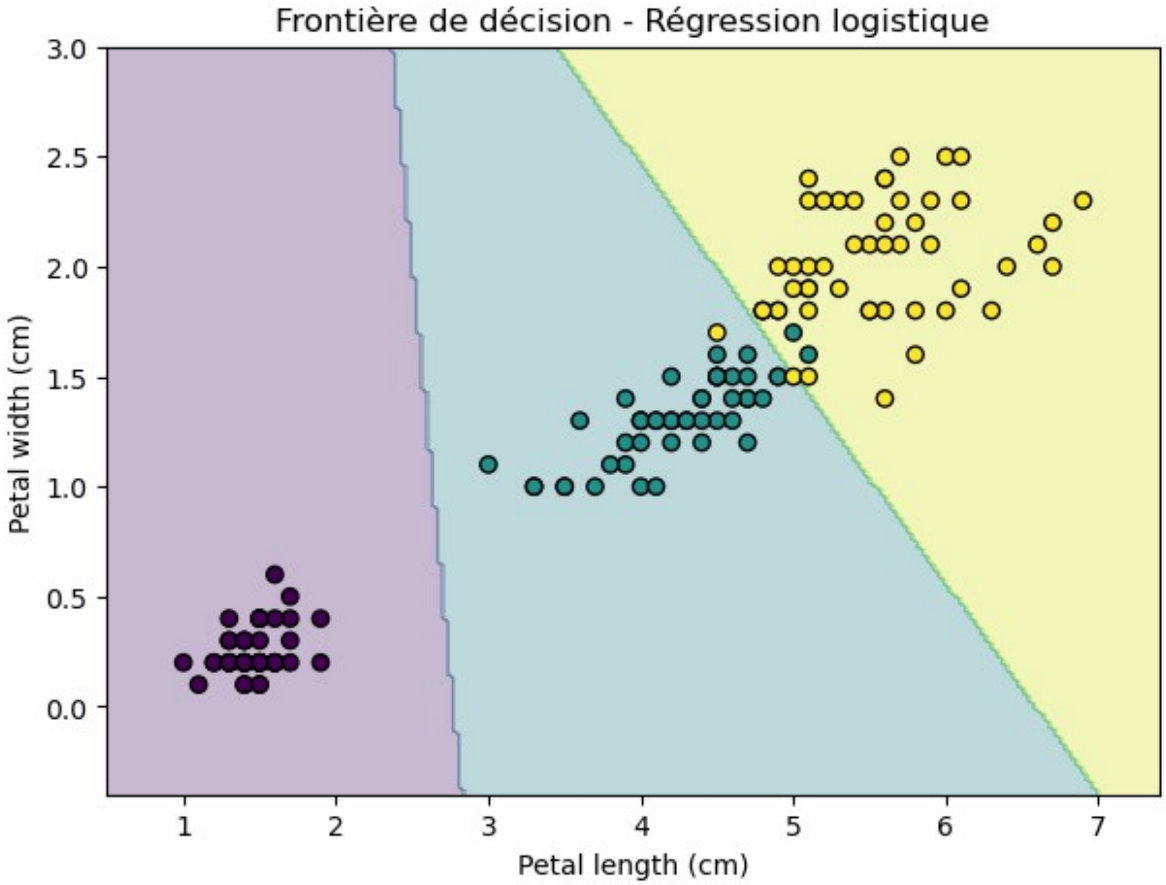


Visualisation de la frontière de décision

```
import numpy as np

# Création d'une grille
x_min, x_max = X_selected.iloc[:, 0].min() - 0.5, X_selected.iloc[:,
0].max() + 0.5
y_min, y_max = X_selected.iloc[:, 1].min() - 0.5, X_selected.iloc[:,
1].max() + 0.5

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(7,5))
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(
    X_selected.iloc[:, 0],
    X_selected.iloc[:, 1],
    c=y,
```

```
    edgecolor='k'
)
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.title("Frontière de décision - Régression logistique")
plt.show()
```

```
C:\Users\t15\anaconda3\Lib\site-packages\sklearn\base.py:439:
UserWarning: X does not have valid feature names, but
LogisticRegression was fitted with feature names
  warnings.warn(
```



Partie 3 : Apprentissage non supervisé – K-means

Charger le fichier "clustring.csv" et afficher ses premières informations (dimensions, aperçu les données)

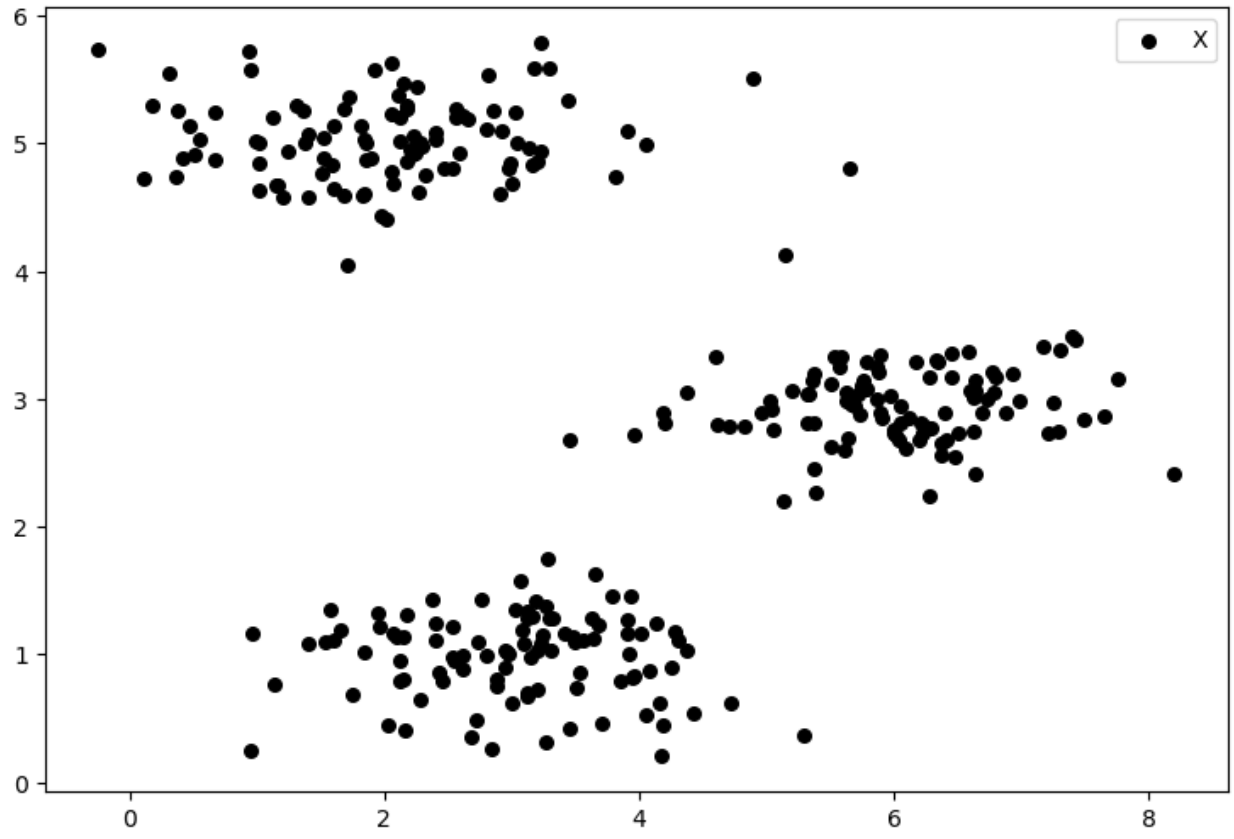
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```python
# Charger le fichier CSV
data = pd.read_csv("clustring.csv", sep=";")

# Vérifier un aperçu des données
data.head()
```

```
          X         Y
0  1.842080  4.607572
1  5.658583  4.799964
2  6.352579  3.290854
3  2.904017  4.612204
4  3.231979  4.939894
```

Visualiser des données : Représentez graphiquement la répartition des valeurs

```python
# Affichage du nuage de points
fig, ax = plt.subplots(figsize=(9,6))
ax.scatter(data['X'], data['Y'],s=30, color='k', label='X')
ax.legend()
plt.show()
```



Implémenter l'algorithme K-means :

Initialiser aléatoirement k centroides

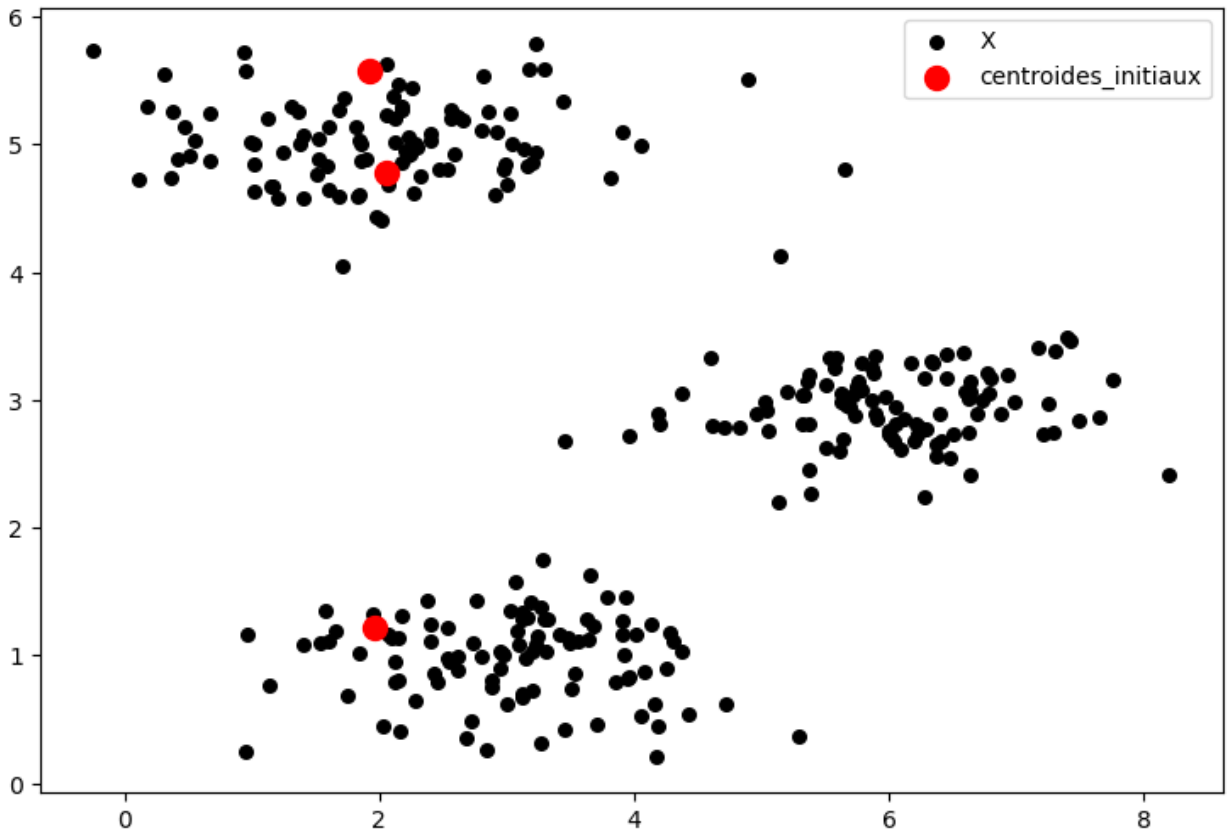
```
# Données (sous forme de numpy array)
X = data[['X','Y']].values

X [:5]
```

```
array([[1.84207953, 4.6075716 ],
       [5.65858312, 4.79996405],
       [6.35257892, 3.2908545 ],
       [2.90401653, 4.61220411],
       [3.23197916, 4.93989405]])
```

```
# Fonction pour sélectionner des points aléatoires comme centroïdes
def init_centroides(X, k):
    m, n = X.shape
    centroides = np.zeros((k, n))   # k points en n dimensions
    idx = np.random.randint(0, m, k) # sélectionner k indices
aléatoires
    for i in range(k):
        centroides[i, :] = X[idx[i], :]
    return centroides

# Initialisation des centroïdes (exemple avec k=3)
centroides_initiaux = init_centroides(X, 3)
print("Centroides initiaux :\n", centroides_initiaux)
```

```
Centroides initiaux :
 [[2.05288518 4.77958559]
 [1.96547974 1.2173076 ]
 [1.91909566 5.57848447]]
```

```
# Affichage des points et des centroïdes initiaux
fig, ax = plt.subplots(figsize=(9,6))
ax.scatter(X[:,0], X[:,1], s=30, color='k', label='X')

ax.scatter(centroides_initiaux[:,0], centroides_initiaux[:,1], s=100,
color='r', label='centroides_initiaux')

ax.legend()
plt.show()
```




```
# Trouver les centres de gravité le plus proche
def centroides_PlusProche(X, centroides):
    m = X.shape[0]
    k = centroides.shape[0]
    idx = np.zeros(m)

    for i in range(m):
        min_distance = np.sum((X[i,:] - centroides[0,:]) ** 2)
        for j in range(k):
            distance = np.sum((X[i,:] - centroides[j,:]) ** 2)
            if distance < min_distance:
                min_distance = distance
                idx[i] = j

    return idx

idx = centroides_PlusProche(X, centroides_initiaux)
idx
```

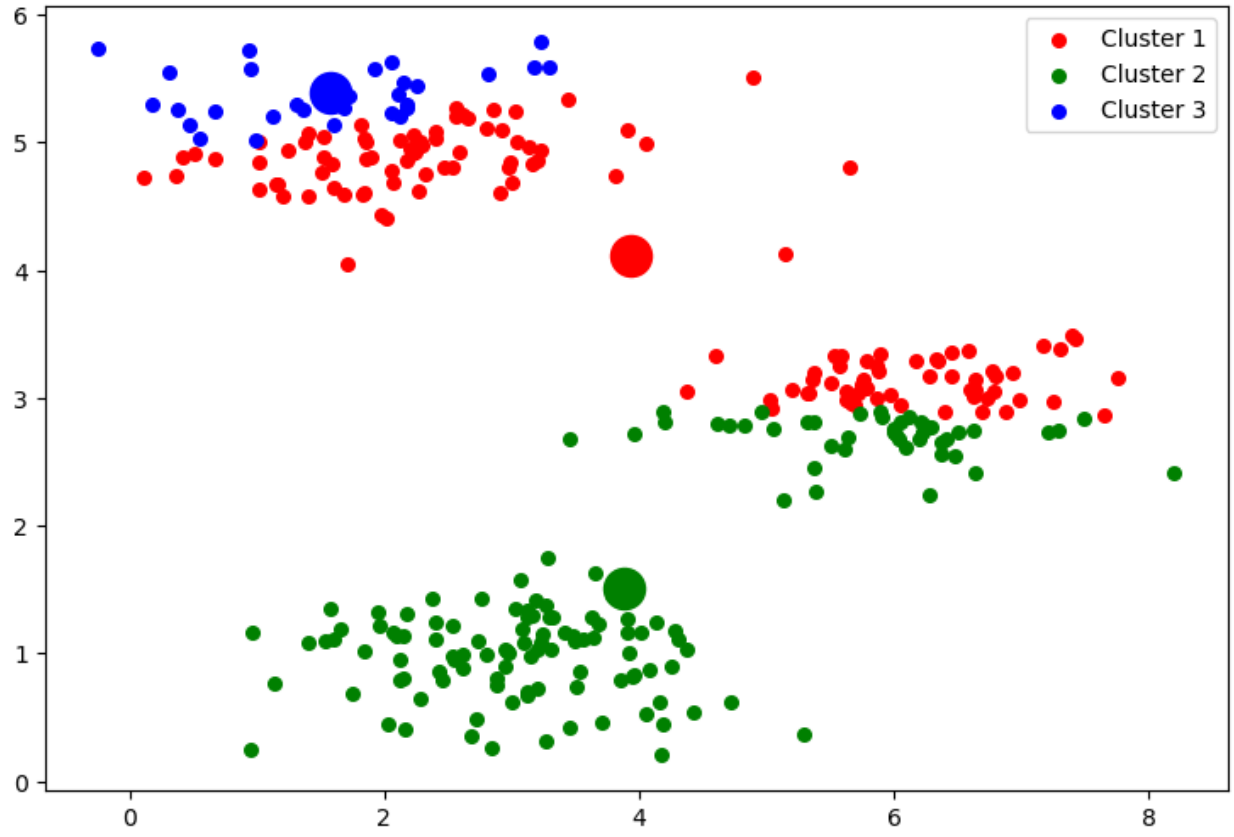
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 2., 2., 0., 2., 0., 0.,
0.,
       0., 0., 0., 0., 0., 2., 2., 0., 2., 0., 2., 2., 0., 0., 0., 0.,
2.,
       0., 0., 0., 2., 0., 0., 2., 0., 2., 2., 0., 2., 0., 0., 0., 0.,
```

```
0.,
       0., 0., 0., 0., 0., 0., 0., 0., 2., 0., 2., 0., 2., 0., 2., 0.,
2.,
       0., 0., 0., 0., 0., 0., 0., 0., 0., 2., 2., 0., 2., 0., 2., 0.,
0.,
       0., 0., 0., 2., 2., 0., 0., 2., 0., 0., 0., 2., 0., 0., 2., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1.,
       1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 0.,
1.,
       0., 0., 0., 0., 0., 1., 0., 0., 1., 1., 1., 1., 0., 0., 0., 0.,
0.,
       1., 1., 0., 1., 0., 1., 0., 0., 1., 1., 1., 0., 0., 0., 0., 0.,
0.,
       0., 1., 0., 1., 0., 0., 1., 0., 0., 1., 0., 1., 1., 0., 1., 0.,
0.,
       1., 0., 1., 0., 1., 1., 0., 0., 1., 0., 1., 1., 1., 1., 0., 1.,
1.,
       0., 1., 0., 1., 0., 0., 1., 0., 0., 0., 0., 1., 1., 1., 0., 1.,
0.,
       0., 1., 1., 0., 0., 1., 0., 0., 1., 1., 2.])
```

```
# calculer les centres de gravité
def compute_centroides(X, idx, k):
    m, n = X.shape
    centroides = np.zeros((k, n))

    for i in range(k):
        indices = np.where(idx == i)
        centroides[i,:] = (np.sum(X[indices,:], axis=1) /
len(indices[0])).ravel()

    return centroides

# Exemple : recalculer les centroïdes avec les assignations idx
centroides = compute_centroides(X, idx, 3)
print("Nouveaux centroïdes :\n", centroides)

# Séparer les clusters
cluster1 = X[idx == 0]
cluster2 = X[idx == 1]
```

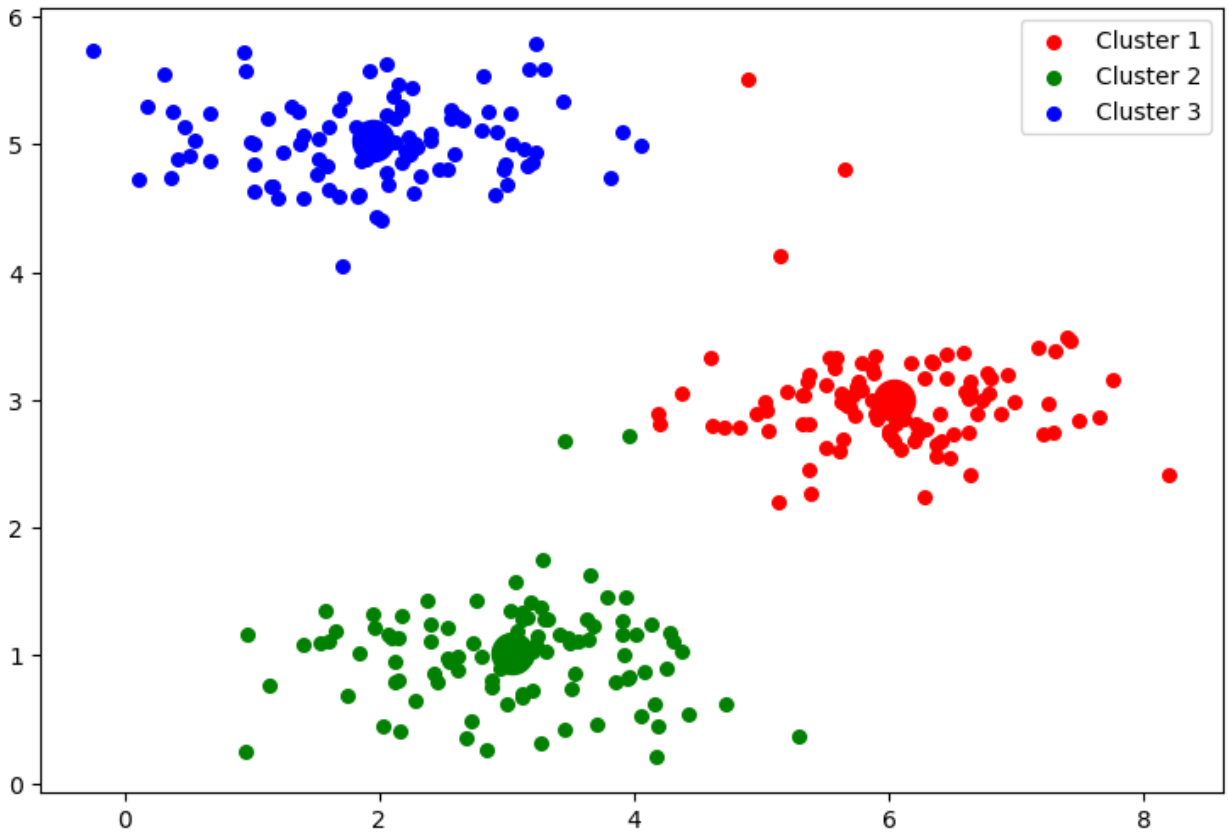
```
cluster3 = X[idx == 2]

# Afficher le résultat
fig, ax = plt.subplots(figsize=(9,6))

ax.scatter(cluster1[:,0], cluster1[:,1], s=30, color='r',
label='Cluster 1')
ax.scatter(centroides[0,0], centroides[0,1], s=300, color='r')

ax.scatter(cluster2[:,0], cluster2[:,1], s=30, color='g',
label='Cluster 2')
ax.scatter(centroides[1,0], centroides[1,1], s=300, color='g')

ax.scatter(cluster3[:,0], cluster3[:,1], s=30, color='b',
label='Cluster 3')
ax.scatter(centroides[2,0], centroides[2,1], s=300, color='b')

ax.legend()
plt.show()
```

```
Nouveaux centroïdes :
 [[3.94088384 4.12096691]
 [3.88391406 1.50456226]
 [1.57023456 5.38311986]]
```



```python
# La fonction k means
def k_means(X, centroides_initiaux, max_iters):
    m, n = X.shape
    k = centroides_initiaux.shape[0]
    idx = np.zeros(m)
    centroides = centroides_initiaux

    for i in range(max_iters):
        idx = centroides_PlusProche(X, centroides) # Trouver les
centres de gravité le plus proche
        centroides = compute_centroides(X, idx, k) # calculer les
centres de gravité

    return idx, centroides

idx, centroides = k_means(X, centroides_initiaux, 10)

# Exemple : recalculer les centroïdes avec les assignations idx
centroides = compute_centroides(X, idx, 3)
print("Nouveaux centroïdes :\n", centroides)

# Séparer les clusters
cluster1 = X[idx == 0]
cluster2 = X[idx == 1]
cluster3 = X[idx == 2]

# Afficher le résultat
fig, ax = plt.subplots(figsize=(9,6))

ax.scatter(cluster1[:,0], cluster1[:,1], s=30, color='r',
label='Cluster 1')
ax.scatter(centroides[0,0], centroides[0,1], s=300, color='r')

ax.scatter(cluster2[:,0], cluster2[:,1], s=30, color='g',
label='Cluster 2')
ax.scatter(centroides[1,0], centroides[1,1], s=300, color='g')

ax.scatter(cluster3[:,0], cluster3[:,1], s=30, color='b',
label='Cluster 3')
ax.scatter(centroides[2,0], centroides[2,1], s=300, color='b')

ax.legend()
plt.show()
```

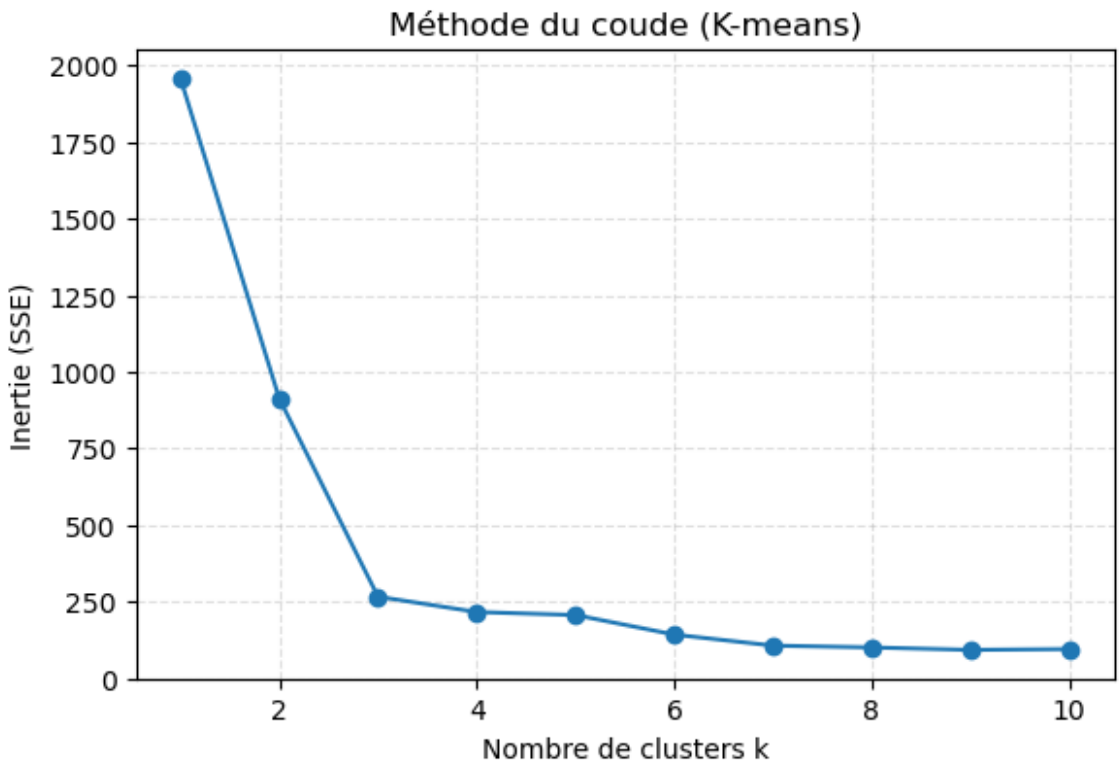
```
Nouveaux centroïdes :
 [[6.03366736 3.00052511]
 [3.04367119 1.01541041]
 [1.95399466 5.02557006]]
```




```
# --- inertie (SSE) robuste : idx entier, à plat ---
def inertie_kmeans(X, idx, centroides):
    """
    Inertie (SSE) = somme des distances euclidiennes au carré
    entre chaque point et le centroïde de son cluster.
    """
    idx = np.asarray(idx, dtype=np.int64).ravel()      # <-
important !
    d2 = np.sum((X - centroides[idx])**2, axis=1)
    return float(np.sum(d2))

# --- courbe Elbow ---
def elbow_kmeans(X, Kmax=10, max_iters=20, seed=42):
    Ks = list(range(1, Kmax + 1))
    inertias = []

    for k in Ks:
        c0 = init_centroides(X, k)          # init des centroïdes
        idx, centroides = k_means(X, c0, max_iters)    # <-- TA
fonction existante
        SSE = inertie_kmeans(X, idx, centroides)
        inertias.append(SSE)

    # Tableau récapitulatif
```

```python
    tab = pd.DataFrame({"k": Ks, "inertie": inertias})
    print(tab)

    # Tracé Elbow
    plt.figure(figsize=(6.5, 4.2))
    plt.plot(Ks, inertias, marker="o")
    plt.xlabel("Nombre de clusters k")
    plt.ylabel("Inertie (SSE)")
    plt.title("Méthode du coude (K-means)")
    plt.grid(True, linestyle="--", alpha=0.4)
    plt.show()

    return Ks, inertias

# --- Exemple d'utilisation ---
Ks, inertias = elbow_kmeans(X, Kmax=10, max_iters=20, seed=42)
```

```
    k      inertie
0   1  1957.654721
1   2   913.319271
2   3   266.658520
3   4   216.589638
4   5   206.772919
5   6   142.952692
6   7   107.024467
7   8   101.005963
8   9    93.052902
9  10    95.552229
```



Comparetion entre les résultats obtenus et ceux du scikit-learn :

```
import warnings
warnings.filterwarnings("ignore")

idx, centroides = k_means(X, centroides_initiaux, 10)
cout = inertie_kmeans(X, idx, centroides)
print("Coût de l'algorithme K-Means implémenté : ", cout)
```

```
Coût de l'algorithme K-Means implémenté :  266.65851965491936
```

```
# Utilisation de K-Means de scikit-learn
from sklearn.cluster import KMeans

kmeans_sklearn = KMeans(n_clusters=3, random_state=42)
kmeans_sklearn.fit(X)
print("Inertie de K-Means scikit-learn : ", kmeans_sklearn.inertia_)
```

```
Inertie de K-Means scikit-learn :  266.6585196549193
```